Mathematical Properties & "Bit Independence"

This is the great filter in competitive programming. Players who only know how to simulate bitwise operations get stuck here.
To solve Div 2 B, C, and D problems consistently, you must stop looking at arrays of integers and start looking at columns of bits.

Here is the mathematical framework that allows you to solve these problems on paper before writing a single line of code.

1. The Principle of Bit Independence

This is the most critical paradigm shift. When dealing with bitwise operators (&, |, ^),
bits do not interact with their neighbors.
There are no "carries" or "borrows" across columns like there are in standard addition or subtraction.

If a problem asks you to maximize the bitwise OR or calculate the sum of XORs across an array of size 10^5,
do not process the integers as whole numbers.

Instead, treat the problem as 30 to 60 completely independent sub-problems:

1. Look only at the 0-th bit of every number in the array. Solve the logic.
2. Look only at the 1-st bit. Solve the logic.
3. Combine the answers at the end by multiplying the result of the i-th bit by 2^i.

2. XOR Algebra (The Magic Operator)

XOR (^) behaves like a highly symmetrical, reversible version of addition.
Memorize these properties, as they are the key to simplifying complex CP equations.

- Self-Inverse (Nilpotency): $x \oplus x = 0$

  (Any number XORed with itself obliterates itself.
  This is used constantly to find "the single non-repeating element" in an array).

- Identity Element: $x \oplus 0 = x$

  (XORing with zero does nothing).

- Commutativity & Associativity: $a \oplus b = b \oplus a$ and $(a \oplus b) \oplus c = a \oplus (b \oplus c)$

  (Order absolutely does not matter. You can rearrange an XOR sum however you want).

- The Reversibility Rule: If $a \oplus b = c$, then you instantly know that $a \oplus c = b$ and $b \oplus c = a$.

  (If you have two pieces of the triangle, you can always find the third.
  This is heavily used in finding missing elements or reversing operations).

3. The Fundamental Equation of Bitwise Math

You will see problems that mix standard arithmetic (+, -) with bitwise operations.
You cannot solve them without this master equation:

$$a + b = (a \oplus b) + 2(a \,\&\, b)$$

Why does this work?

When you add two numbers in binary:

1. a ^ b gives you the addition without the carries. (If one bit is 1 and the other is 0, the sum is 1. If both are 1, it becomes 0).

2. a & b tells you exactly where the carries occur (where both bits are 1).

3. Because a carry moves one position to the left, we multiply the carry bits by 2 (which is a left shift: << 1).

Corollary: Because a & b tracks the overlapping 1s, and a | b tracks all 1s, another valid identity is:

$$a + b = (a \,\&\, b) + (a \mid b)$$

4. Prefix XOR for Range Queries

In standard math, if you want the sum of a subarray from index L to R, you use a prefix sum array: Sum(L, R) = P[R] - P[L-1].
Because XOR is its own inverse ($x \oplus x = 0$), you can do the exact same thing without needing subtraction.

1. Build a prefix XOR array: P[i] = P[i-1] ^ A[i]
2. To find the XOR of the range [L, R], simply compute: P[R] ^ P[L-1]

The elements from index 0 to L - 1 exist in both P[R] and P[L - 1].
When you XOR them together, they perfectly cancel each other out, leaving only the elements from L to R.

5. The MSB (Most Significant Bit) Greedy Property

When a problem asks you to maximize a bitwise result (especially XOR or OR), you must build your answer greedily from the highest bit (e.g., bit 30) down to the lowest bit (bit 0).

The Golden Rule: The i-th bit is strictly more valuable than all the bits below it combined.
In binary, 1000...0 (a single 1 followed by k zeros) is exactly 1 greater than 0111...1 (a zero followed by k ones).

Therefore, if you have a choice in an algorithm to secure a 1 in the 20th bit, you always take it, even if taking it forces bits 19 down to 0 to all become 0.
You never sacrifice a higher bit for lower bits.